

同濟大學

TONGJI UNIVERSITY

《数据采集与集成技术》

实验报告

实验名称 爬取天气预报数据与网页结构分析

实验成员 ()

日期 2025 年 11 月 8 日

1、实验目的

本次实验旨在通过以中国天气网为具体目标，引导学习者深入掌握网络数据采集与分析的全流程。核心目标在于深刻理解并能实际操作静态网页与动态网页这两种不同网页类型的数据爬取方法。实验要求学习者不仅要熟练运用如 BeautifulSoup 这类库来解析和提取结构相对固定的静态 HTML 内容，还要掌握使用 Selenium 等浏览器自动化工具，来模拟用户行为并获取由 JavaScript 动态加载和渲染的数据。此外，实验强调了对网页前端结构（DOM）的分析能力，要求能够准确定位目标数据、理解动态内容的加载机制（如 AJAX 请求）。最终，实验旨在打通从数据获取、数据存储（使用数据库或文件）到数据可视化（绘制图表）的完整链路，从而将理论知识与实践应用相结合，全面提升应对复杂网络数据采集项目的综合能力。

2、实验内容

本实验从基础的静态网页信息提取入手，逐步过渡到需要运用浏览器自动化技术才能处理的复杂动态网页。在此基础上，实验进一步要求对采集到的数据进行持久化存储和图形化展示，最后通过对比分析，巩固和深化对不同网页技术及其爬取策略的理解，从而全面构建网络数据采集的核心技能。

为实现此目标，实验内容涵盖了从服务器端数据处理到客户端页面渲染与交互的完整开发链路，具体划分如下：

2.1 静态网页爬取与 DOM 结构分析

该任务聚焦于基础的静态网页处理能力。您需要针对中国天气网上包含上海 9 月份历史天气数据的固定页面，使用像 BeautifulSoup 这样的解析库，编写爬虫程序来提取网页中结构化的信息，具体包括每日的日期、最高与最低气温、天气概况以及风力风向。同时，为了加深对网页结构的理解，还需要分析并输出该页面的 DOM 结构树，清晰地展示出目标数据在网页代码中的具体位置和层级关系。

2.2 动态网页爬取与结构分析

本任务将难度提升至动态网页的数据采集，要求使用 Selenium 等浏览器自动化工具来应对由 JavaScript 动态加载和渲染的内容。可以从实时天气、逐小时预报或生活指数这三类动态页面中任选其一，在成功抓取其动态数据的同时，更要深入分析这些数据是如何加载的，例如通过侦测和分析 XHR/AJAX 请求来找到背后真正的数据源，并识别出动态内容在 DOM 中被插入的区域与时机。

2.3 数据存储与可视化

这个任务是数据应用的起点，旨在将前序任务中采集到的原始数据转化为更易于管理和理解的形式。需要将爬取到的天气数据存入到 SQLite 或 MySQL 等关系型数据库中，或整理成结构化的文件。在此基础上，您将利用这些数据进行可视化实践：一是根据上海 9 月份的历史数据绘制温度变化的曲线图，以展现气温随时间波动的趋势；二是通过采集全国省会城市的天气数据，绘制一幅天气状况对比的柱状图，实现数据的横向比较。

2.4 DOM 结构对比分析

作为实验的收尾，最后一个任务侧重于理论的总结。它要求我对前述静态页面和动态页面的 DOM 结构进行一次深入的对比分析，阐明它们在结构上的本质区别。核心在于，我需要结合自

己的实践过程，具体分析动态内容（如 AJAX 返回的数据）是如何通过 JavaScript 脚本被精确地“注入”或“更新”到现有网页的 DOM 树中，从而真正从原理上理解动态网页的工作机制。

3、实验步骤

3.1 静态网页爬取与 DOM 结构分析

3.1.1 爬取天气网中上海市 9 月份天气页面

本任务的目标是编写一个 Python 脚本，自动从天气网（<https://lishi.tianqi.com>）的历史天气页面抓取特定月份（2025 年 9 月）上海市的每日天气数据，并对网页的结构进行分析。

在开始编码前，首先需要准备好实验环境。这个脚本依赖于几个核心的 Python 库，我们在代码的开头将它们全部导入：

- **requests**：用于向目标网站发送 HTTP 请求，获取网页的 HTML 源代码，这是爬虫获取数据的基础。
- **BeautifulSoup**：一个强大的 HTML/XML 解析库，它能将复杂的 HTML 文档转换成一个易于操作的 Python 对象树，是数据提取的关键。
- **Pandas**：用于将提取出的数据整理成结构化的 DataFrame，方便在控制台清晰地预览和检查。

随后发送 HTTP 请求并获取网页内容。我在主程序块中定义我要爬取的目标页面 <https://lishi.tianqi.com/shanghai/202509.html>，为了防止被网站服务器识别为爬虫并拒绝访问，我们设置了 headers，特别是 User-Agent 字段。这会让我们请求看起来像是来自一个真实的浏览器。最后使用 `requests.get(url, headers=headers)` 发送 GET 请求，`response.raise_for_status()` 会检查请求是否成功（状态码为 200）。如果遇到网络问题或服务器错误，程序会捕获异常并打印错误信息，确保了脚本的健壮性。

```
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
    like Gecko) Chrome/91.0.4472.124 Safari/537.36'
}
```

在获取到网页内容后，我要解析 HTML 并提取核心数据，这是整个任务的核心。首先创建 BeautifulSoup 对象将获取到的 HTML 文本交给 BeautifulSoup 进行解析，生成一个可以被程序理解和遍历的 soup 对象。

其次，定位数据容器，通过分析网页源代码，我们发现所有历史天气数据都被包含在一个 class 属性为 thrui 的无序列表 标签内，因此，代码精确定位到这个总容器。这是 DOM 分析的第一步，也是后续数据提取的基础。

随后，我要遍历并提取每日数据，获取容器内所有的列表项 标签，每一个 都代表一天的数据，通过一个 for 循环，我们逐一处理每一天的 元素，在每个 内部，日期、最高温、最低温、天气状况和风力风向分别被包裹在 5 个连续的 <div> 标签中。代码获取这 5 个 <div>。

最后，我按顺序从这 5 个 <div> 中提取文本内容，并进行简单的数据清洗（如去除“℃”符号），然后将这些信息存入一个字典中。所有字典最终被添加到一个名为 weather_data 的列表中。

3.1.2 输出所爬取页面的 DOM 结构树

为了完成实验要求的 DOM 结构分析，代码特别设计了 generate_dynamic_dom_tree 函数。该函数以数据提取时定位到的核心容器 <ul class="thrui"> 为起点，利用 BeautifulSoup 的 .parents 属性，动态地向上追溯并格式化输出一条直达 <body> 标签的清晰 DOM 路径，从而宏观地展示了目标数据在页面中的位置。

更进一步，它还向下解构了容器内一个典型的 子元素，精确地将其内部的各个 <div> 与实

验所要求的日期、温度、天气状况等具体数据项一一对应起来,通过这种由外到内、由宏观到微观的展示方式,完美地完成了 DOM 结构分析的任务。

3.2 动态网页爬取与结构分析

3.2.1 使用 Selenium 爬取中国天气网中上海实时天气信息

本任务的核心挑战在于处理由 JavaScript 动态加载和渲染的网页内容,目标网页是中国天气网(<http://www.weather.com.cn/weather/101020100.shtml>)与 3.1 不同,我们不能简单地通过一次 HTTP 请求获取全部信息,因为关键数据(如逐小时天气预报)是在浏览器加载页面后,通过执行 JavaScript 代码才生成或填充到页面中的。因此,我们必须使用像 Selenium 这样的浏览器自动化工具来模拟真实的用户浏览器环境。

首先进行环境配置与初始化 Selenium WebDriver,首先导入所有必需的 Selenium 库,然后通过指定本地 chromedriver.exe 的路径并配置一系列 Chrome 选项来准备浏览器设置,其中“--headless”参数的设置尤为关键,它允许浏览器在后台静默运行,极大地提升了自动化脚本的执行效率。最后,通过 webdriver.Chrome()实例化一个浏览器驱动对象,该对象将作为我们后续所有页面操作的执行者。

在拥有了可控的浏览器实例后,我使用 driver.get()命令指示它导航至指定的上海天气预报 URL。由于该页面的核心数据(逐小时天气)是通过 JavaScript 异步加载的,不能立即开始抓取。因此,脚本中加入了一个 time.sleep(2)的短暂等待,这个停顿给予了页面足够的时间去执行其内部的 JavaScript 脚本,完成从服务器请求数据并将其渲染到页面上的过程,确保了后续数据提取的可靠性。

然后我寻找数据最原始的来源。通过浏览器开发者工具分析发现,逐小时的天气数据在渲染成图表前,被存储在一个名为 hour3data 的全局 JavaScript 变量中。因此,脚本采取了一种更为高效和稳定的方法:使用 driver.execute_script 命令,直接在浏览器当前页面环境中执行 JavaScript 代码来获取这个变量的值,从而绕过了复杂的 HTML 解析,直接从源头获得了结构化的原始数据。

从 JavaScript 环境中获取的原始数据需要经过解析和清洗才能使用。脚本首先从返回的字典中提取出存储 24 小时预报信息的列表,然后遍历该列表。由于列表中的每一项都是一个逗号分隔的字符串,拆分后按预定的顺序提取出时间、天气状况、温度等关键信息,最后将这些零散的信息重新组织成一个由多个字典构成的列表,使数据变得规整且易于后续操作。

3.2.2 分析动态加载数据的来源

在发起 HTTP 请求获取未经渲染的原始 HTML 后,这份代码随后被 BeautifulSoup 解析成一个可供程序检查的对象,它完整地保留了页面加载前的所有静态结构和内嵌脚本。

第一,查找 JavaScript 动态插入的目标元素,脚本在静态 HTML 中搜索一个具有特定 ID (curve)的元素,这个 ID 是预先通过浏览器开发者工具确定的动态内容容器。关键的发现是,在原始 HTML 中,这个容器元素内部是空的。这一现象有力地证明了它是一个“占位符”,其作用就是等待页面的 JavaScript 脚本在运行时,将根据 hour3d_data 变量动态创建的图表和表格等内容填充进去,从而完成页面的动态渲染。

第二,分析 AJAX 请求的数据源,在这一关键步骤中,我们的目标是揭示逐小时天气预报数据的真正来源。脚本会深入到 HTML 的内联<script>标签中,通过搜索关键词 hour3data 来定位存储数据的 JavaScript 变量。一旦找到,便利用正则表达式精确地提取出赋给该变量的、完整的 JSON 格式字符串。最后,通过尝试将此字符串解析为 Python 字典进行验证。

第三,识别定时刷新的数据区域,最后一步分析将目光投向了页面上其他可能存在的动态行为,特别是那些需要定时更新的模块,例如顶部的“实况天气”。脚本通过在内联代码中搜索与实时数据相关的特定变量名(如 observe24h_data 或 dataSK),以及用于发起 AJAX 请求的核心技术关键词 XMLHttpRequest,来寻找线索。

3.2.3 提取数据并输出动态生成部分的 DOM 结构

虽然我们通过捷径获取了数据,但实验要求仍然需要分析动态内容在 DOM 中的位置。我使

用显式等待定位元素：为了找到承载这些天气数据的 HTML 容器（ID 为 `curve` 的图表区域），代码使用了 `WebDriverWait`。它会智能地等待最多 5 秒，直到这个动态生成的元素在页面上可见为止。这是一种比 `time.sleep()` 更可靠的等待方式。然后生成 DOM 树，一旦定位成功，就可以从这个动态容器开始，向上追溯并打印出其在 DOM 树中的完整路径，从而完成对动态生成部分 DOM 结构的分析。

3.3 数据存储与可视化

3.3.1 将爬取的数据存入数据库

为实现数据的长久保存和后续分析，脚本将处理好的数据存入 SQLite 数据库。首先连接数据库并创建一张专门的表（如果不存在的话），该表结构设计了 UNIQUE 约束来防止重复录入。录入函数中最核心的部分是对非标准日期时间字符串（如“08 日 23 时”）的智能解析，它能够正确处理跨月甚至跨年的情况，将其转换为标准的数据库时间格式。最终，利用 `executemany` 将所有数据一次性、高效地批量插入数据库中。

3.3.2 绘制上海市 9 月份温度变化曲线图

该实验步骤旨在将存储在 SQLite 数据库中的原始天气数据转化为一张信息丰富的可视化图表。整个流程首先通过 Pandas 库连接数据库并高效地将上海 9 月份的历史天气数据加载为 DataFrame，随即进入一个至关重要的数据清洗与预处理阶段，该阶段将文本格式的日期和温度数据 meticulously 转换为可供计算的 datetime 和数值类型。在此基础上，脚本利用 matplotlib 库精心绘制了一张包含最高与最低温度两条曲线的图表，并通过添加数据标签、填充温差区域、设置中文字体标题和图例等一系列美化操作，最终生成并展示了一张直观且易于解读的每日温度变化趋势图。

3.3.3 绘制全国省会城市当天天气情况柱状图

首先通过 requests 库遍历并爬取中国天气网下设的华北、东北等多个地域分区的 HTML 页面（例如 <https://www.weather.com.cn/textFC/hb.shtml>），接着利用 BeautifulSoup 对返回的源代码进行精细化解析，通过识别表格中特有的 `rowsPan` 行标识，巧妙地定位并提取出全国各省会城市当日的最高与最低气温数据。在数据收集完毕后，脚本调用强大的 pyecharts 库，将所有城市名作为 X 轴，最高与最低气温作为两个数据系列，生成一个交互式的柱状图，并通过设置标题、旋转坐标轴标签以及添加数据缩放滑块等选项进行深度美化，最终将这张信息丰富、操作友好的可视化图表渲染成一个独立的 HTML 文件，以便在浏览器中进行查看和分析。

3.4 DOM 结构对比分析

第一步是获取并审查页面的初始 DOM 结构，旨在捕获两种网页在任何 JavaScript 执行前的“原始”状态。分别使用 requests 库（而非 Selenium）来请求静态历史天气页面和动态实时天气页面的 URL，获取它们未经浏览器渲染的 HTML 源代码。然后，仔细检查这两份源代码，对于静态页面，确认所有天气数据（如具体日期和温度）都作为文本直接存在于 HTML 标签内；对于动态页面，则要找到其核心内容容器（如 `<div id="curve"></div>`），并验证其在原始代码中是空的，以此作为后续对比分析的基础。

第二步是对比并总结 DOM 结构的核心差异，在获取了两种页面的初始 HTML 后，本步骤将进行直接的对比分析。总结并记录两者在内容呈现上的根本不同：静态页面的 DOM 是“内容与结构一体化”的，下载完成时即是最终的完整形态；而动态页面的初始 DOM 则是一个不完整的“结构骨架”，其核心数据缺失，相应的容器仅作为等待被填充的“占位符”。通过这一对比，可以得出静态 DOM 是服务器端渲染的最终产物，而动态 DOM 是客户端渲染的起点的结论。

第三步是在静态代码中定位动态内容的数据源，此步骤的目标是揭示动态页面数据的隐藏位置。需要在之前获取的动态页面原始 HTML 代码中进行“侦查”，通过搜索我们在任务二中

发现的关键 JavaScript 变量名（如 hour3data），来定位到定义这个变量的<script>标签。分析并确认该变量的值是一个包含了所有逐小时预报数据的 JSON 对象，从而得出结论：该页面的数据源并非通过后续 AJAX 请求获得，而是以 JSON 格式直接嵌入在页面的内联脚本中，与 HTML 骨架一同被浏览器加载。

第四步是利用开发者工具分析 JavaScript 的 DOM 插入过程，最后一步旨在彻底弄清动态内容是如何被“注入”到页面中的。需要在浏览器中打开动态天气页面，并启动开发者工具。通过在“Sources”（源代码）面板中搜索关键变量名 hour3data 或目标容器 IDcurve，找到负责渲染的 JavaScript 代码。通过阅读这段代码，分析并理解其核心逻辑：它首先通过 document.getElementById("curve")获取 DOM 中的空容器，然后遍历 hour3data 数据，在内存中动态创建新的 HTML 元素，最后调用.appendChild()或.innerHTML 等方法，将这些新创建的、已填充数据的元素批量插入到空容器中，完成页面的最终渲染。

4、实验结果

4.1 静态网页爬取与 DOM 结构分析

脚本成功访问了目标 URL 并精确地爬取了 2025 年 9 月上海市的历史天气数据，并以清晰的表格形式展示了包括日期、最高/最低温、天气状况及风力在内的多维度信息。

```
正在从 https://lishi.tianqi.com/shanghai/202509.html 爬取数据...
数据爬取成功！

爬取到的数据预览：
      日期  最高温  最低温  天气状况  风力风向
0  2025-09-01  星期一  32℃  27℃  小雨  东北风 2级
1  2025-09-02  星期二  31℃  26℃  中雨  西风 3级
2  2025-09-03  星期三  34℃  27℃  小雨  东南风 3级
3  2025-09-04  星期四  34℃  27℃  小雨  东南风 3级
4  2025-09-05  星期五  35℃  27℃  晴  东南风 3级
5  2025-09-06  星期六  36℃  28℃  小雨  东南风 3级
6  2025-09-07  星期日  37℃  28℃  多云  东北风 3级
7  2025-09-08  星期一  36℃  28℃  小雨  东南风 3级
8  2025-09-09  星期二  33℃  27℃  中雨  西北风 3级
9  2025-09-10  星期三  29℃  26℃  多云  东北风 3级

=====
```

图 1 2025 年 9 月上海市的历史天气数据爬取截图

更为关键的是，脚本通过动态分析生成了目标数据在页面中的 DOM 结构树，该结构树直观地揭示了数据从<body>标签开始，经过层层嵌套，最终定位到核心数据容器<ul class="thru">的完整路径。分析进一步深入到单个数据条目的内部，明确了各项具体数据（如日期、温度）与其对应的<div>标签的精确关系，从而完美达成了对静态网页数据进行提取和结构分析的实验目标。

还可以进行检索，示例如下：

根据本次爬取结果动态生成的DOM结构树：

```
<body>
├── <div class="main clearfix">
│   ├── <div class="main_left inleft">
│   │   ├── <div class="tian_three">
│   │   │   └── <ul class="thru1"> (← **数据父容器**)
│   │   │       ├── <li> (← **单日数据容器**)
│   │   │           ├── <div class="th200"> (a) 日期
│   │   │           ├── <div class="th140"> (b) 最高温
│   │   │           ├── <div class="th140"> (b) 最低温
│   │   │           ├── <div class="th140"> (c) 天气状况
│   │   │           └── <div class="th140"> (d) 风力风向
│   │   └── ... (后续 <li> 结构与此相同)
```

图 2 2025 年 9 月上海市的历史天气数据 DOM 结构分析截图

本次 DOM 结构分析结果清晰而精确地揭示了目标数据在网页中的嵌套层级与组织方式，分析从页面的<body>标签出发，通过层层递进成功定位到了包含所有天气信息的父容器<ul class="thru1">，并进一步解构了代表单日数据的元素。最关键的是，它精确地将日期、温度、天气状况等具体数据项与内部的各个<div>子标签建立了一一对应的关系。

4.2 动态网页爬取与结构分析

脚本利用 Selenium 自动化浏览器，精准地从动态加载的上海天气页面中提取了核心的逐小时天气预报数据。脚本成功模拟浏览器执行了页面的 JavaScript，并从一个名为 hour3data 的 JS 变量中直接获取了未来 24 小时的预报数据。提取的数据经过解析和格式化后，在控制台输出如下：

```
正在访问目标网页：http://www.weather.com.cn/weather/101020100.shtml
正在提取逐小时天气数据...

--- 上海市未来24小时逐小时天气预报 ---

时间          | 天气状况      | 温度      | 风向      | 风力
-----
08日 23时     | 小雨          | 17℃      | 西风      | <3级
09日 02时     | 小雨          | 17℃      | 西风      | <3级
09日 05时     | 小雨          | 17℃      | 西北风    | <3级
09日 08时     | 小雨          | 17℃      | 北风      | <3级
09日 11时     | 多云          | 19℃      | 北风      | 3-4级
09日 14时     | 多云          | 19℃      | 北风      | 3-4级
09日 17时     | 小雨          | 17℃      | 北风      | 3-4级
09日 20时     | 阴            | 17℃      | 北风      | 3-4级

数据爬取成功！
```

图 3 上海市实时天气数据爬取截图

随后对加载动态数据的来源进行分析，分析结果显示，逐小时天气预报的数据并非通过后续的 AJAX 请求从服务器获取，而是直接以一个完整的 JSON 对象的形式，硬编码在了一个名为 hour3data 的 JavaScript 变量中，并嵌入在页面的 11 个内联<script>标签之一。脚本成功提取并验证了这个 JSON 对象的结构，确认了其包含 'ld' 等关键数据。同时，分析也定位到了 ID 为 curve 的 HTML <div>元素，它作为动态内容的“目标容器”。这一发现有力地证明了该模块采用的是“数据预置，客户端渲染”的模式：浏览器一次性加载包含数据（hour3data 变量）和空框架（<div

id="curve">) 的 HTML, 然后由本地 JavaScript 读取变量中的数据, 动态生成 HTML 内容并填充到指定容器中。

与逐小时预报不同, 分析结果表明, 页面顶部的“实况天气”模块采用了另一种动态更新机制。脚本在内联代码中成功识别出了两个关键线索: 一是存储过去 24 小时实况数据的变量 `observe24h_data`, 二是用于发起异步网络请求的核心对象 `XMLHttpRequest`。这两个线索共同指向了一种“定时 AJAX 刷新”的模式: 页面会通过 JavaScript 代码, 定时地、在后台向服务器发起 AJAX 请求以获取最新的实况天气数据, 然后更新页面顶部的相应区域, 而无需用户刷新整个页面。

```
✓ 成功提取到 hour3data 的第一个 JSON 对象。
{"1d":["08日23时,n07,小雨,17℃西风,<3级,0","09日02时,n07,小雨,17℃西风,<3级,0","09日05时,n07,小雨,17℃西北风,<3级,0",
"09日08时,d07,小雨,17℃北风,<3级,3","09日11时,d01,多云,19℃北风,3-4级,1","09日14时,d01,多云,19℃北风,3-4级,3","09日17
时,d07,小雨,1 ... [内容省略]
✓ 接下来, 对该字符串进行验证分析...
✓ 成功将字符串解析为 Python 字典进行验证。
  - 验证成功: 找到了 '1d' 键, 包含 8 条数据。

===== 任务二: 查找 JS 动态插入的元素 =====
✓ 找到了包含 'hour3data' 数据的 <script> 标签 (数据源已就绪)。
✓ 找到了准备接收动态内容的目标容器: <div id="curve" class="curve_livezs">
  - 注意: 容器内部已有内容, 可能是静态的框架。

[结论]: 页面同时提供了【数据源('hour3data'变量)】和【目标容器(id='curve')】。JS脚本负责读取数据源, 生成HTML, 并填充到
目标容器中。

===== 任务三: 识别定时刷新的数据区域 =====
✓ 找到了包含过去24小时实况数据的变量 'observe24h_data'。这是页面顶部“实况天气”模块的数据源。
✓ 找到了使用 'XMLHttpRequest' (发起XHR/AJAX请求) 的内联代码。
```

图 4 加载动态数据来源分析结果截图

脚本首先通过 `id='curve'` 定位器, 成功捕获到了承载动态生成内容的目标容器。随后, 通过从该目标元素向上追溯其所有父节点, 脚本清晰地描绘出了该容器在整个 DOM 树中的完整层级路径。该路径显示, 目标容器 `<div id="curve" class="curve_livezs">` 位于一个 ID 为 7d 的 `<div>` 内, 并依次向上嵌套在 `left-div`、`left fl`、`con today clearfix` 等 `<div>` 中, 最终归属到 `<body>` 标签下。这份直观的 DOM 结构图, 为我理解 JavaScript 脚本是如何在页面的特定“坐标”上精确插入动态内容, 提供了决定性的证据。

```
正在使用最终正确的定位器定位容器: id='curve'...
成功定位到目标容器!

===== 动态内容DOM结构追溯 =====
起始元素: <div id="curve" class="curve_livezs">
-----
<body>
  <div class="con today clearfix">
    <div class="left fl">
      <div class="left-div">
        <div id="7d" class="c7d">
          <div id="curve" class="curve_livezs"> <-- [目标容器]
=====
```

图 5 上海市逐小时天气数据 DOM 结构分析截图

4.3 数据存储与可视化

首先我将将通过静态和动态爬虫获取的两类天气数据持久化存储到了 SQLite 数据库中, 为后续的数据分析和可视化任务奠定了坚实的基础。


```
--- 正在查询数据库 'weather_history.db' 中的表 'shanghai_weather_202509' ---
查询成功! 数据如下:
```

	id	record_date	max_temp	min_temp	weather	wind
0	1	2025-09-01 星期一	32	27	小雨	东北风 2级
1	2	2025-09-02 星期二	31	26	中雨	西风 3级
2	3	2025-09-03 星期三	34	27	小雨	东南风 3级
3	4	2025-09-04 星期四	34	27	小雨	东南风 3级
4	5	2025-09-05 星期五	35	27	晴	东南风 3级
5	6	2025-09-06 星期六	36	28	小雨	东南风 3级
6	7	2025-09-07 星期日	37	28	多云	东北风 3级
7	8	2025-09-08 星期一	36	28	小雨	东南风 3级
8	9	2025-09-09 星期二	33	27	中雨	西北风 3级
9	10	2025-09-10 星期三	29	26	多云	东北风 3级

图 6 2025 年 9 月上海市的历史天气数据存储数据库截图

```
--- 正在查询数据库 'weather_forecast.db' 中的表 'shanghai_hourly' ---
查询成功! 数据如下:
```

	id	forecast_datetime	weather_condition	temperature_celsius	wind_direction	wind_force
0	1	2025-11-08 02:00:00	小雨	18	东南风	<3级
1	2	2025-11-08 05:00:00	小雨	18	东风	<3级
2	3	2025-11-08 08:00:00	小雨	18	东南风	<3级
3	4	2025-11-08 11:00:00	小雨	20	南风	<3级
4	5	2025-11-08 14:00:00	多云	21	西风	<3级
5	6	2025-11-08 17:00:00	小雨	18	西风	<3级
6	7	2025-11-08 20:00:00	小雨	17	西风	<3级
7	8	2025-11-08 23:00:00	小雨	17	西风	<3级
8	9	2025-11-09 02:00:00	小雨	17	西风	<3级
9	10	2025-11-09 05:00:00	小雨	17	西北风	<3级
10	11	2025-11-09 08:00:00	小雨	17	北风	<3级
11	12	2025-11-09 11:00:00	多云	19	北风	3-4级
12	13	2025-11-09 14:00:00	多云	19	北风	3-4级
13	14	2025-11-09 17:00:00	小雨	17	北风	3-4级

图 7 上海市逐小时天气数据存储数据库截图

随后，实验成功地将存储在数据库中的结构化数据转化为了两种直观、信息丰富的可视化图表。

脚本成功连接到 weather_history.db 数据库，使用 Pandas 库读取了上海市 9 月份的历史天气数据。经过必要的数据库清洗和类型转换后，利用 Matplotlib 库绘制了一张温度变化曲线图。

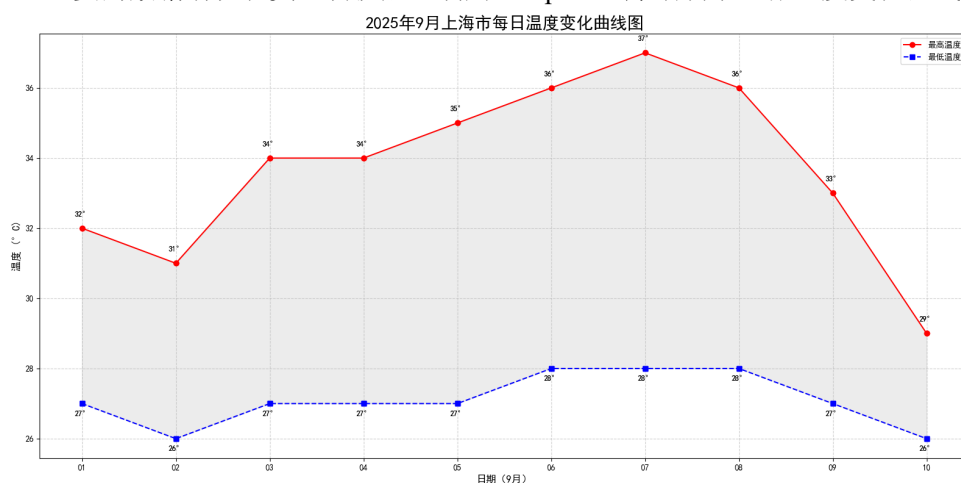


图 8 2025 年 9 月上海市温度变化曲线图

脚本通过遍历中国天气网的多个地域分区页面，实时爬取了全国所有省会城市（包括港澳台）当天的最高与最低气温数据。随后，利用 Pyecharts 库将这些数据动态渲染成一个交互式的柱状图。

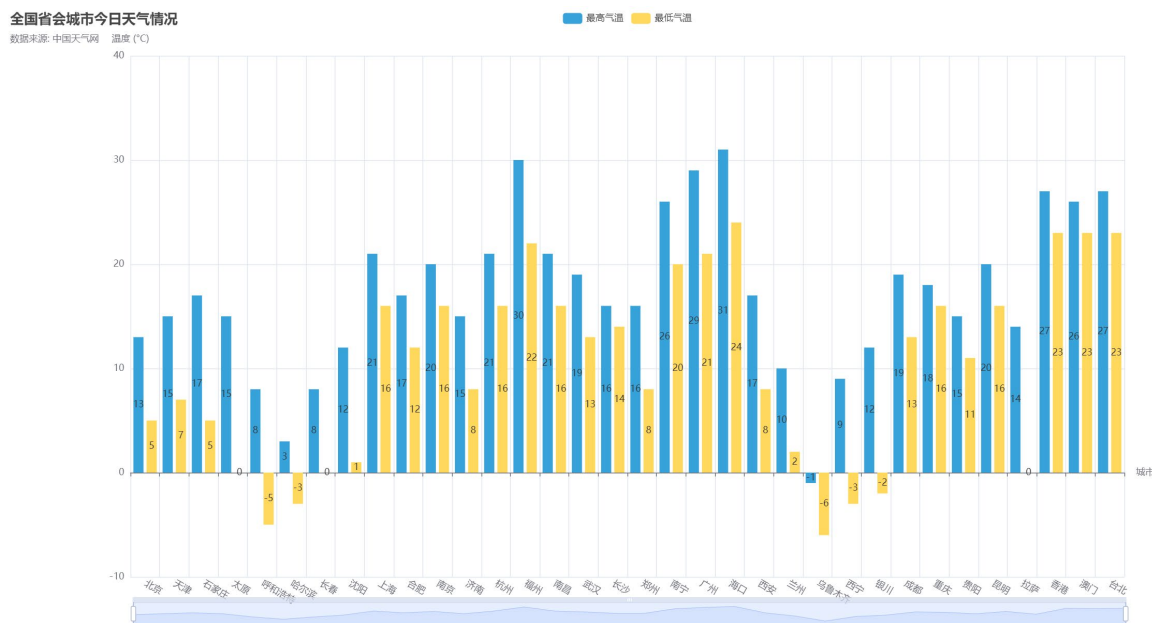


图9 全国省会城市当天天气情况柱状图

本次可视化实验不仅成功运用了 Matplotlib 和 Pyecharts 这两个主流的 Python 绘图库, 还根据不同的数据特性和分析需求, 选择了最合适的图表类型。

4.4 DOM 结构对比分析

通过对比分析, 静态与动态页面的 DOM 结构差异显著: 静态页面的 DOM 在服务器端已完全生成, 其数据内容与 HTML 标签结构紧密耦合, 浏览器下载后即可获得一个包含所有信息的完整、最终的文档结构。而动态页面的初始 DOM 则是一个不完整的内容“骨架”, 其核心数据区域 (如<div id="curve">) 在原始代码中为空占位符, 这种“内容与结构分离”的设计, 决定了其最终页面形态必须依赖于客户端的二次处理。

动态内容的插入机制是一个在客户端完成的渲染过程: 浏览器首先加载包含空 DOM “骨架”和预置在<script>标签内的数据源 (如 hour3data 变量) 的 HTML 页面。当浏览器加载完页面后, 一个或多个 JavaScript 脚本开始执行。这些脚本首先会读取 hour3data 变量中的 JSON 数据, 然后在内存中遍历这些数据, 并根据预设的模板动态地创建出如图表 (<canvas>)、表格 (<table>) 等一系列新的 HTML 元素。最后, 脚本通过 document.getElementById('curve')找到目标容器, 并使用.appendChild()或.innerHTML 等 DOM 操作方法, 将这些在内存中新鲜创建的、包含完整内容的元素, “注入”到那个原本为空的<div>容器中, 从而完成页面的最终渲染, 让用户看到动态生成的内容。

5、实验总结

本次“爬取天气预报数据与网页结构分析”实验我深入实践并全面掌握了网络数据采集与分析的核心流程, 达到了预期的实验目的。。

我熟练运用了 Python 生态中的多个关键库: 通过 Requests 和 BeautifulSoup, 我成功实现了对静态网页的高效解析与数据提取; 面对由 JavaScript 动态渲染的网页, 我掌握了使用 Selenium 自动化浏览器来模拟用户行为, 并从中获取动态内容的核心技巧。此外, 我还通过 Pandas 进行了高效的数据规整, 利用 SQLite 完成了数据的持久化存储, 并分别使用 Matplotlib 和 Pyecharts 将处理后的数据转化为了直观的曲线图和交互式柱状图, 打通了从数据采集、处理、存储到可视化

的完整链路。

本次实验最核心的收获在于深刻理解了静态网页与动态网页在 DOM 结构和内容加载机制上的本质区别。通过亲手实践和对比分析，我清晰地认识到：静态页面的 DOM 是“内容与结构一体”的，在服务器端一次性生成；而动态页面的初始 DOM 则是一个“内容与结构分离”的骨架，其最终形态依赖于客户端 JavaScript 读取数据源后对 DOM 进行的动态构建和注入。我对 hour3data 变量的分析，揭示了“数据预置，客户端渲染”的常见动态模式，同时对 XMLHttpRequest 的识别也让我理解了“定时 AJAX 刷新”的工作原理。

在问题解决与实践能力方面，我不仅成功编写了能够应对不同网页类型的爬虫脚本，还通过生成 DOM 结构树、分析 JS 变量等方式，锻炼了独立分析网页背后技术实现的能力。整个实验过程不仅是对编程技能的考验，更是对分析问题、解决问题综合能力的全面提升。

附录

附录中是本次实验的代码。

static_weather.py

```
import requests
from bs4 import BeautifulSoup, Tag
import pandas as pd
import sqlite3 # 导入 SQLite 库

def get_element_signature(element):
    """
    为 HTML 元素生成一个简洁的签名，包含标签名、ID 和类。
    """
    if not isinstance(element, Tag):
        return str(element)
    signature = f"<{element.name}"
    element_id = element.get('id')
    if element_id:
        signature += f' id="{element_id}"'
    element_class = element.get('class')
    if element_class:
        signature += f' class="{ " ".join(element_class)}"'
    signature += ">"
    return signature

def generate_dynamic_dom_tree(target_element):
    """
    从目标元素开始，向上追溯并动态生成到<body>的 DOM 结构树。
    """
    print("\n" + "="*50 + "\n")
    print("根据本次爬取结果动态生成的 DOM 结构树:")
    if not target_element:
        print("错误：未能定位到目标元素，无法生成 DOM 树。")
        return
    path_to_body = []
```

```

current_element = target_element
for parent in current_element.parents:
    path_to_body.append(parent)
    if parent.name == 'body':
        break
path_to_body.reverse()
indentation = "    "
for i, element in enumerate(path_to_body):
    print(f"{indentation * i}|— {get_element_signature(element)}")
    print(f"{indentation * len(path_to_body)}|—
{get_element_signature(target_element)} (← **数据父容器**)")
    first_li = target_element.find('li')
    if first_li:
        li_indent = indentation * (len(path_to_body) + 1)
        print(f"{li_indent}|")
        print(f"{li_indent}|— {get_element_signature(first_li)} (← **单日数
据容器**)")
        divs = first_li.find_all('div')
        div_indent = li_indent + "    "
        if len(divs) == 5:
            print(f"{div_indent}|— {get_element_signature(divs[0])} (a) 日期
")
            print(f"{div_indent}|— {get_element_signature(divs[1])} (b) 最高
温")
            print(f"{div_indent}|— {get_element_signature(divs[2])} (b) 最低
温")
            print(f"{div_indent}|— {get_element_signature(divs[3])} (c) 天气
状况")
            print(f"{div_indent}|— {get_element_signature(divs[4])} (d) 风力
风向")
        print(f"{li_indent}|")
        print(f"{li_indent}|— ... (后续 <li> 结构与此相同)")
    print("\n")
def scrape_shanghai_weather(url):
    """
    从指定的 tianqi.com 网址爬取上海的历史天气数据。
    """
    try:
        headers = {
            'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36'
        }
        response = requests.get(url, headers=headers)
        response.raise_for_status()
        soup = BeautifulSoup(response.content, 'html.parser')
        weather_list_container = soup.find('ul', class_='thru1')
    
```

```

        if not weather_list_container:
            print("错误: 未能找到天气数据的容器 (<ul class='thruil'>)。网页结构可能已更改。")
            return None, None
        weather_data = []
        for day_item in weather_list_container.find_all('li'):
            details = day_item.find_all('div')
            if len(details) == 5:
                date = details[0].get_text(strip=True)
                # 清洗数据, 去掉'°C'符号, 便于存入数据库
                max_temp = details[1].get_text(strip=True).replace('°C', '')
                min_temp = details[2].get_text(strip=True).replace('°C', '')
                weather_condition = details[3].get_text(strip=True)
                wind = details[4].get_text(strip=True)
                weather_data.append({
                    "日期": date,
                    "最高温": max_temp + '°C',
                    "最低温": min_temp + '°C',
                    "天气状况": weather_condition,
                    "风力风向": wind
                })
            return weather_data, weather_list_container
    except requests.exceptions.RequestException as e:
        print(f"网络请求错误: {e}")
        return None, None
    except Exception as e:
        print(f"发生了一个未知错误: {e}")
        return None, None

def save_to_sqlite(data, db_name="weather_history.db",
table_name="shanghai_202509"):
    """
    将天气数据列表存入 SQLite 数据库。
    Args:
        data (list): 包含天气数据字典的列表。
        db_name (str): 数据库文件名。
        table_name (str): 数据表名。
    """
    if not data:
        print("没有数据可存入 SQLite。")
        return
    # 使用 with 语句可以确保数据库连接在使用后自动关闭
    try:
        with sqlite3.connect(db_name) as conn:
            cursor = conn.cursor()
            # 1. 创建表 (如果表已存在, 则不执行任何操作)
            # - record_date 设置为 UNIQUE, 防止同一天的数据被重复插入

```

装

订

线


```
create_table_sql = f"""
CREATE TABLE IF NOT EXISTS {table_name} (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    record_date DATE NOT NULL UNIQUE,
    max_temp INTEGER,
    min_temp INTEGER,
    weather TEXT,
    wind TEXT
);
"""

cursor.execute(create_table_sql)
# 2. 准备要插入的数据
# 将字典列表转换为元组列表，以匹配 SQL 的 '?' 占位符
rows_to_insert = [
    (item['日期'], item['最高温'], item['最低温'], item['天气状况'],
item['风力风向'])
    for item in data
]
# 3. 插入数据
# 使用 INSERT OR IGNORE，如果日期已存在（违反 UNIQUE 约束），则忽略
该条插入，不会报错
insert_sql = f"""
INSERT OR IGNORE INTO {table_name} (record_date, max_temp, min_temp,
weather, wind)
VALUES (?, ?, ?, ?, ?);
"""

# executemany 可以高效地执行多次插入
cursor.executemany(insert_sql, rows_to_insert)
# with 语句会自动调用 conn.commit()
print(f"\n 数据已成功存入 SQLite 数据库 '{db_name}' 的 '{table_name}'
表中。")

# 打印本次操作实际插入的行数
print(f"本次操作新增了 {cursor.rowcount} 条记录。")

except sqlite3.Error as e:
    print(f"SQLite 操作失败: {e}")
# --- 主程序入口 ---
if __name__ == "__main__":
    target_url = "https://lishi.tianqi.com/shanghai/202509.html"
    print(f"正在从 {target_url} 爬取数据...")
    scraped_data, target_element = scrape_shanghai_weather(target_url)
    if scraped_data and target_element:
        print("\n 数据爬取成功! \n")
        # 使用 pandas 展示数据
        df = pd.DataFrame(scraped_data)
        print("爬取到的数据预览:")
        print(df)
```

```
# 动态生成 DOM 树
generate_dynamic_dom_tree(target_element)
# 将数据存入 SQLite 数据库
save_to_sqlite(scraped_data, table_name="shanghai_weather_202509")
else:
    print("\n 数据爬取失败。请检查 URL 或网络连接。")
```

dynamic_weather.py

```
import time
import json
import sqlite3
from datetime import datetime, date, timedelta
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException,
NoSuchElementException

def get_element_signature_selenium(element):
    """为 Selenium 的 WebElement 生成一个简洁的签名。"""
    signature = f"<{element.tag_name}"
    element_id = element.get_attribute('id')
    if element_id:
        signature += f' id="{element_id}"'
    element_class = element.get_attribute('class')
    if element_class:
        signature += f' class="{element_class}"'
    signature += ">"
    return signature

def generate_dynamic_dom_tree_selenium(target_element):
    """
    从目标 Selenium 元素开始，向上追溯并动态生成到<body>的 DOM 结构树。
    这个版本会打印出每个节点的简洁签名（标签名、ID、类名）。
    """
    if not target_element:
        print("错误：传入的目标元素无效。")
        return
    print("\n" + "="*20 + " 动态内容 DOM 结构追溯 " + "="*20)
    print(f"起始元素: {get_element_signature_selenium(target_element)}")
    print("-" * (42 + len(" 动态内容 DOM 结构追溯 ")))
    path_to_body = []
    current_element = target_element
    # 循环向上追溯父元素，直到 body 或 html 标签
```

```

while current_element.tag_name not in ['body', 'html']:
    path_to_body.append(current_element)
    try:
        # 使用 XPath '..' 来获取父元素
        parent_element = current_element.find_element(By.XPATH, '..')
        current_element = parent_element
    except NoSuchElementException:
        # 如果找不到父元素（极少见，除非 DOM 结构异常），则停止
        print("警告：在追溯过程中某个元素没有父节点，已到达 DOM 树顶端。")
        break

# 将 body 标签也加入路径中
if current_element.tag_name == 'body':
    path_to_body.append(current_element)
# 反转列表，从 body 开始打印
path_to_body.reverse()
# 打印 DOM 树结构
indent_level = 0
for element in path_to_body:
    indent = " " * indent_level
    signature = get_element_signature_selenium(element)
    # 判断当前元素是否为我们最开始的目标元素
    if element == target_element:
        print(f"{indent}{signature} <-- [目标容器]")
    else:
        print(f"{indent}{signature}")
    indent_level += 1
print("\n" + "="*50 + "\n")

def save_hourly_forecast_to_sqlite(data, db_name="weather_forecast.db",
table_name="shanghai_hourly"):
    """
    将逐小时天气预报数据存入 SQLite 数据库。
    """
    if not data:
        print("没有数据可存入 SQLite。")
        return
    try:
        with sqlite3.connect(db_name) as conn:
            cursor = conn.cursor()
            create_table_sql = f"""
            CREATE TABLE IF NOT EXISTS {table_name} (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                forecast_datetime TEXT NOT NULL UNIQUE,
                weather_condition TEXT,
                temperature_celsius INTEGER,
                wind_direction TEXT,
                wind_force TEXT
            """

```

装

订

线

```

);
"""

cursor.execute(create_table_sql)
rows_to_insert = []
today = date.today()
for item in data:
    time_str = item['time'] # e.g., "08 日 23 时"
    # 1. 更稳健地解析时间和日期
    try:
        # 替换掉所有非数字字符，然后根据长度判断
        cleaned_str = ''.join(filter(str.isdigit, time_str))
        if len(cleaned_str) > 2: # 格式是 "DDHH", e.g., "0823"
            forecast_day = int(cleaned_str[:-2])
            forecast_hour = int(cleaned_str[-2:])
        else: # 格式是 "HH", e.g., "23"
            forecast_day = today.day
            forecast_hour = int(cleaned_str)
    except (ValueError, IndexError):
        print(f"警告: 无法解析时间字符串 '{time_str}', 跳过此条记录。")

    continue # 跳过这一条错误数据
    # 2. 更精确地处理跨月/跨年
    current_year = today.year
    current_month = today.month
    # 如果预报的日期小于今天的日期（例如，在 30 号爬取到了 1 号的数据），
    # 说明月份+1
    if forecast_day < today.day:
        if current_month == 12:
            current_month = 1
            current_year += 1
        else:
            current_month += 1
    # 3. 构建完整的日期时间对象，避免无效日期
    try:
        full_datetime_obj = datetime(current_year, current_month,
forecast_day, forecast_hour)
        full_datetime_str =
full_datetime_obj.strftime('%Y-%m-%d %H:%M:%S')
    except ValueError:
        print(f"警告: 无法构建有效日期
{current_year}-{current_month}-{forecast_day}, 跳过。")
        continue
    temperature = int(item['temperature'].replace('°C', ''))
    rows_to_insert.append((
        full_datetime_str,
        item['weather_condition'],

```

```

        temperature,
        item['wind_direction'],
        item['wind_force']
    ))
    insert_sql = f"""
    INSERT OR IGNORE INTO {table_name}
    (forecast_datetime, weather_condition, temperature_celsius,
    wind_direction, wind_force)
    VALUES (?, ?, ?, ?, ?);
    """
    cursor.executemany(insert_sql, rows_to_insert)
    print(f"\n 数据已成功存入 SQLite 数据库 '{db_name}' 的 '{table_name}'
表中。")
    print(f"本次操作新增了 {cursor.rowcount} 条记录。")
except sqlite3.Error as e:
    print(f"SQLite 操作失败: {e}")
if __name__ == "__main__":
    CHROMEDRIVER_PATH = r"D:\chromedriver-win64\chromedriver.exe"
    chrome_options = Options()
    chrome_options.add_argument("--headless")
    chrome_options.add_argument("--disable-gpu")
    chrome_options.add_argument("--no-sandbox")
    chrome_options.add_argument("--disable-dev-shm-usage")
    chrome_options.add_argument("--window-size=1920,1080")
    service = Service(executable_path=CHROMEDRIVER_PATH)
    driver = webdriver.Chrome(service=service, options=chrome_options)
    url = "http://www.weather.com.cn/weather/101020100.shtml"
    try:
        print(f"正在访问目标网页: {url}")
        driver.get(url)
        time.sleep(2)
        print("正在提取逐小时天气数据...")
        hourly_data_js = driver.execute_script("return hour3data;")
        data_extraction_successful = False
        if not hourly_data_js:
            print("未能成功提取到 hour3data 变量。")
        else:
            hourly_forecast_list = hourly_data_js.get('1d')
            if not hourly_forecast_list:
                print("未能找到'1d'键对应的逐小时预报数据。")
            else:
                collected_data = []
                for forecast_str in hourly_forecast_list:
                    parts = forecast_str.split(',')
                    collected_data.append({
                        "time": parts[0],

```



```

        "weather_condition": parts[2],
        "temperature": parts[3],
        "wind_direction": parts[4],
        "wind_force": parts[5]
    })
    print("\n--- 上海市未来 24 小时逐小时天气预报 ---\n")
    print(f"{'时间':<12} | {'天气状况':<10} | {'温度':<8} | {'风向':<10} | {'风力':<10}")
    print("-" * 60)
    for item in collected_data:
        print(f"{item['time']:<12} | {item['weather_condition']:<10} | {item['temperature']:<8} | {item['wind_direction']:<10} | {item['wind_force']:<10}")
        print("\n 数据爬取成功! ")
        data_extraction_successful = True
        save_hourly_forecast_to_sqlite(collected_data)
    if data_extraction_successful:
        try:
            wait = WebDriverWait(driver, 5)
            locator = (By.ID, "curve")
            print(f"\n 正在使用最终正确的定位器定位容器: id='curve'...")
            target_container_element =
wait.until(EC.visibility_of_element_located(locator))
            print("成功定位到目标容器! ")
            generate_dynamic_dom_tree_selenium(target_container_element)
# 可以取消注释来显示 DOM 树
        except TimeoutException:
            print("\n 错误: 虽然数据已提取, 但在指定时间内未能定位到动态内容容器(id='curve')。")
        except Exception as e:
            print(f"爬取过程中发生错误: {e}")
        finally:
            print("关闭浏览器...")
            driver.quit()

```

analyze_weather_page.py

```

import requests
from bs4 import BeautifulSoup
import re
import json

def find_data_source(soup_obj):
    """
    分析 HTML, 找出 'hour3data' 变量的来源。
    """
    print("\n" + "="*20 + " 任务一: 分析动态加载数据的来源 " + "="*20)

```

```
# 查找所有不含 'src' 属性的 <script> 标签（即内联脚本）
all_scripts = soup_obj.find_all('script', src=None)
all_scripts_text = "\n".join([s.get_text() for s in all_scripts])
print(f"页面中共找到 {len(all_scripts)} 个内联 <script> 标签。")
found_in_script = False
for script in all_scripts:
    # 检查<script>标签内是否有文本内容，并且是否包含目标变量
    if script.string and 'hour3data' in script.string:
        print("☑ 找到包含 hour3data 的 <script> 段落。")
        # ☑ 提取 hour3data 内的第一个 {...}
        match = re.search(r"var\s+hour3data\s*=\s*(\{.*?\});",
all_scripts_text, re.DOTALL)
        if match:
            json_string = match.group(1)
            # ☑ 手动括号配对，确保只取第一个完整 JSON 对象
            brace_count = 0
            end_index = 0
            for i, ch in enumerate(json_string):
                if ch == '{':
                    brace_count += 1
                elif ch == '}':
                    brace_count -= 1
                    if brace_count == 0:
                        end_index = i + 1
                        break
            json_string = json_string[:end_index]
            # ☑ 尝试修复乱码（编码错误）
            try:
                json_string_fixed =
json_string.encode('latin1').decode('utf-8')
            except UnicodeEncodeError:
                json_string_fixed = json_string # 如果不需要修复则保留原文
            print(json_string_fixed)
            print("\n☑ 成功提取到 hour3data 的第一个 JSON 对象。")
            print(json_string_fixed[:200] + " ... [内容省略]")
            print("☑ 接下来，对该字符串进行验证分析...")
            try:
                data = json.loads(json_string_fixed)
                print("☑ 成功将字符串解析为 Python 字典进行验证。")
                if '1d' in data:
                    print(f"    - 验证成功：找到了 '1d' 键，包含
{len(data['1d'])} 条数据。")
                else:
                    print("    - 注意：未找到 '1d' 键，但 JSON 结构有效。")
            except json.JSONDecodeError as e:
                print("✗ JSON 解析失败：", e)
```

```

        else:
            print("✗ 未能通过正则表达式提取出完整的 hour3data 对象。")
            found_in_script = True
            break # 找到后退出循环
    if not found_in_script:
        print("✗ 未在任何内嵌 <script> 标签中找到 'hour3data' 变量。")
def find_dynamic_insertion_elements(soup_obj):
    """
    查找 JS 动态插入内容的“数据源”和“目标容器”。
    """
    print("\n" + "="*20 + " 任务二：查找 JS 动态插入的元素 " + "="*20)
    # 1. 验证数据源的存在
    data_script_exists = bool(soup_obj.find(string=re.compile("var
hour3data")))
    if data_script_exists:
        print("☑ 找到了包含 'hour3data' 数据的 <script> 标签 (数据源已就绪)。")
    else:
        print("✗ 未找到 'hour3data' 数据源。")
    # 2. 验证目标容器的存在
    target_container = soup_obj.find(id="curve")
    if target_container:
        print(f"☑ 找到了准备接收动态内容的目标容器: <{target_container.name}
id=\"{target_container.get('id')}\">
class=\"{target_container.get('class')[0]}\">")
    # 3. 检查容器当前是否为空
    if not target_container.find_all(recursive=False):
        print("    - 容器在原始 HTML 中是空的，这完全符合“等待 JS 动态插入内容”
的特征。")
    else:
        print("    - 注意：容器内部已有内容，可能是静态的框架。")
    else:
        print("✗ 未找到 ID 为 'curve' 的目标容器。")
    if data_script_exists and target_container:
        print("\n[结论]：页面同时提供了【数据源('hour3data'变量)】和【目标容器
(id='curve')】。JS 脚本负责读取数据源，生成 HTML，并填充到目标容器中。")
def find_periodic_refresh_clues(soup_obj):
    """
    从 HTML 中寻找与定时刷新相关的线索。
    """
    print("\n" + "="*20 + " 任务三：识别定时刷新的数据区域 " + "="*20)
    # 1. 寻找实时数据相关的变量线索
    # `observe24h_data` 存储了过去 24 小时的实况数据，是定时刷新的目标
    realtime_data_script = soup_obj.find(string=re.compile("var
observe24h_data"))
    if realtime_data_script:

```

```

    print("☑ 找到了包含过去 24 小时实况数据的变量 'observe24h_data'。这是页面顶部“实况天气”模块的数据源。")
else:
    # 有时实况数据是通过一个叫 dataSK 的变量加载的
    realtime_data_script_sk = soup_obj.find(string=re.compile("var dataSK"))
    if realtime_data_script_sk:
        print("☑ 找到了包含实况数据的变量 'dataSK'。这是页面顶部“实况天气”模块的数据源。")
    else:
        print("✗ 未在内联脚本中找到明显的实况数据变量（如 'observe24h_data' 或 'dataSK'）。")
# 2. 寻找发起异步请求的代码线索
xhr_script = soup_obj.find(string=re.compile("XMLHttpRequest"))
if xhr_script:
    print("☑ 找到了使用 'XMLHttpRequest'（发起 XHR/AJAX 请求）的内联代码。")
else:
    print("✗ 在内联脚本中未直接找到使用 'XMLHttpRequest' 的代码（可能封装在外部 JS 文件中）。")
if __name__ == "__main__":
    target_url = "http://www.weather.com.cn/weather/101020100.shtml"
    print(f"正在从 URL 获取 HTML 内容: {target_url}\n")
    try:
        # 发送网络请求，并伪装成浏览器
        headers = {
            'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36'
        }
        response = requests.get(target_url, headers=headers, timeout=10)
        # 如果请求失败（如 404），则抛出异常
        response.raise_for_status()
        # 使用 BeautifulSoup 解析获取到的 HTML
        soup = BeautifulSoup(response.text, 'html.parser')
        # 依次执行三个分析任务
        find_data_source(soup)
        find_dynamic_insertion_elements(soup)
        find_periodic_refresh_clues(soup)
    except requests.exceptions.RequestException as e:
        print(f"网络请求失败: {e}")
    except Exception as e:
        print(f"发生未知错误: {e}")

```

plot_weather.py

```

import sqlite3
import pandas as pd
import matplotlib.pyplot as plt

```

```
import matplotlib.font_manager as fm

def plot_temperature_curve(db_name="weather_history.db",
                           table_name="shanghai_weather_202509"):
    """
    从 SQLite 数据库中读取指定表格的天气数据，并绘制温度变化曲线图。
    """
    try:
        print(f"正在连接到数据库: {db_name}")
        with sqlite3.connect(db_name) as conn:
            query = f"SELECT record_date, max_temp, min_temp FROM {table_name}"
            ORDER BY record_date;"
            print(f"正在执行查询: {query}")
            df = pd.read_sql_query(query, conn)
        except sqlite3.OperationalError as e:
            print(f"数据库操作错误: {e}")
            print(f"请确保数据库文件 '{db_name}' 和数据表 '{table_name}' 已存在且名称正确。")
            return
        except Exception as e:
            print(f"发生未知错误: {e}")
            return
        if df.empty:
            print("错误: 未能从数据库中获取到任何数据，无法绘图。")
            return
        print("\n 成功获取原始数据，预览如下:")
        print(df.head())
        print("\n 原始数据类型:")
        df.info()
        print("\n--- 正在进行数据清洗和类型转换 ---")
        # 1. 从 'record_date' 文本中提取出日期部分
        # 例如，从 "2025-09-01 星期一" 中提取 "2025-09-01"
        print("正在清理 'record_date' 列，移除多余的文本...")
        df['record_date'] = df['record_date'].str.split(' ').str[0]
        print("\n 清理后的 'record_date' 列预览:")
        print(df.head())
        # 2. 现在，可以安全地转换日期列了
        df['record_date'] = pd.to_datetime(df['record_date'], format='%Y-%m-%d',
            errors='coerce')
        # 3. 转换温度列（作为安全检查，即使现在类型正确）
        df['max_temp'] = pd.to_numeric(df['max_temp'], errors='coerce')
        df['min_temp'] = pd.to_numeric(df['min_temp'], errors='coerce')
        # 4. 删除可能在转换过程中产生的任何坏数据行
        df.dropna(inplace=True)
        if df.empty:
            print("错误: 数据清洗后没有剩余的有效数据，无法绘图。请检查数据库内容。")
```

```

        return
    print("\n 数据清洗和转换完成，处理后数据类型如下:")
    df.info()
    # --- 3. 开始绘图 ---
    print("\n 正在生成温度曲线图...")
    try:
        plt.rcParams['font.sans-serif'] = ['SimHei']
        plt.rcParams['axes.unicode_minus'] = False
    except:
        print("警告：未能设置中文字体'SimHei'。")
    plt.figure(figsize=(16, 8))
    days = df['record_date'].dt.strftime('%d')
    plt.plot(days, df['max_temp'], marker='o', linestyle='-', color='red',
label='最高温度')
    plt.plot(days, df['min_temp'], marker='s', linestyle='--', color='blue',
label='最低温度')
    # --- 4. 美化图表 ---
    plt.fill_between(days, df['max_temp'], df['min_temp'], color='gray',
alpha=0.15)
    # 将温度值转换为整数再显示，避免出现 .0
    for a, b in zip(days, df['max_temp']):
        plt.text(a, b + 0.3, f'{int(b)}°', ha='center', va='bottom', fontsize=9)
    for a, b in zip(days, df['min_temp']):
        plt.text(a, b - 0.2, f'{int(b)}°', ha='center', va='top', fontsize=9)
    plt.title('2025 年 9 月上海市每日温度变化曲线图', fontsize=20)
    plt.xlabel('日期 (9 月)', fontsize=12)
    plt.ylabel('温度 (°C)', fontsize=12)
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    # 5. 显示图表
    print("图表已生成，即将显示...")
    plt.show()
if __name__ == "__main__":
    plot_temperature_curve(
        db_name="weather_history.db",
        table_name="shanghai_weather_202509"
    )

```

city_weather.py

```

import requests
from bs4 import BeautifulSoup
from pyecharts import options as opts
from pyecharts.charts import Bar
from pyecharts.globals import ThemeType

```



```
def scrape_and_generate_chart():
    """
    爬取全国省会城市天气数据并生成柱状图（修正版）。
    """

    # 基础 URL
    base_url = "https://www.weather.com.cn"
    # 包含全国所有区域的页面路径
    regions = [
        '/textFC/hb.shtml', # 华北
        '/textFC/db.shtml', # 东北
        '/textFC/hd.shtml', # 华东
        '/textFC/hz.shtml', # 华中
        '/textFC/hn.shtml', # 华南
        '/textFC/xb.shtml', # 西北
        '/textFC/xn.shtml', # 西南
        '/textFC/gat.shtml' # 港澳台
    ]
    # 用于存储所有省会城市数据的列表
    cities = []
    high_temps = []
    low_temps = []
    print("正在开始从中国天气网爬取数据...")
    # 遍历所有区域页面
    for region_path in regions:
        url = base_url + region_path
        print(f"正在处理: {url}")
        try:
            # 发送 HTTP 请求，并设置 headers 模拟浏览器访问
            headers = {
                'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36'
            }
            response = requests.get(url, headers=headers)
            response.encoding = 'utf-8' # 确保正确解码中文
            if response.status_code == 200:
                soup = BeautifulSoup(response.text, 'html.parser')
                # 找到包含天气数据的 div (只找第一个, 即当天的数据)
                conMidtab = soup.find('div', class_='conMidtab')
                if not conMidtab:
                    continue
                # 找到所有省份的表格
                tables = conMidtab.find_all('table')
                for table in tables:
                    # 寻找包含省份信息的行
                    rows = table.find_all('tr')
                    # 从第二行开始遍历 (跳过表头)
```

```

        for i in range(2, len(rows)):
            cells = rows[i].find_all('td')
            # 确保行中有足够的数据
            if len(cells) < 8:
                continue
            # 省会城市通常是每个省份表格的第一行数据
            is_capital = "rowsPan" in str(cells[0])
            # 如果是港澳台页面，每个城市都是一个"省份"，所以也适用
            if is_capital:
                city = cells[1].text.strip()
                # 如果城市尚未被记录，则添加数据
                if city not in cities:
                    high_temp = int(cells[4].text.strip())
                    low_temp = int(cells[7].text.strip())
                    cities.append(city)
                    high_temps.append(high_temp)
                    low_temps.append(low_temp)
                    print(f"  成功获取: {city}, 最高温: {high_temp}°C, 最低温: {low_temp}°C")
                # 如果已经记录过，就直接跳过，不再抓取后面几天的数据
                else:
                    continue
            else:
                print(f"请求失败，状态码: {response.status_code}")
        except requests.RequestException as e:
            print(f"请求发生错误: {e}")
    print("\n 数据爬取完毕！正在生成图表...")
    # 使用 Pyecharts 绘制柱状图
    bar = (
        Bar(init_opts=opts.InitOpts(theme=ThemeType.LIGHT, width="1600px",
            height="800px"))
        .add_xaxis(cities)
        .add_yaxis("最高气温", high_temps)
        .add_yaxis("最低气温", low_temps)
        .set_global_opts(
            title_opts=opts.TitleOpts(
                title="全国省会城市今日天气情况",
                subtitle=f"数据来源: 中国天气网"
            ),
            tooltip_opts=opts.TooltipOpts(
                is_show=True,
                trigger="axis",
                axis_pointer_type="cross"
            ),
            xaxis_opts=opts.AxisOpts(
                name="城市",

```

```

axislabel_opts=opts.LabelOpts(rotate=-30) # X 轴标签旋转，防止
重叠
    ),
    yaxis_opts=opts.AxisOpts(
        name="温度 (°C)",
    ),
    datazoom_opts=[ # 添加滑动条用于区域缩放
        opts.DataZoomOpts(
            is_show=True,
            type_="slider",
            range_start=0,
            range_end=40, # 默认显示约前 40%的数据
        ),
    ],
)
# 生成 HTML 文件
output_file = "全国省会城市天气柱状图.html"
bar.render(output_file)
print(f"图表已成功生成！请在浏览器中打开文件：{output_file}")
if __name__ == '__main__':
    scrape_and_generate_chart()

```

装

订

线